Detect Arbitrage

AlgoExpert · Very Hard · Graphs

The problem

- Exchange-rate matrix; is there a cycle of trades yielding profit?
- Take -log(rate) edge weights and look for a negative cycle with Bellman-Ford.

Intuition

- Multiplying rates becomes adding minus logs, so profit means a negative cycle, found by Bellman-Ford.

Brute force vs optimal

- Try all cycles → exponential
- −log weights + Bellman–Ford → $O(n^3)$
- Floating-point tolerance

Approach

- 1. Weights are minus log of the rate
- 2. Relax all edges n minus one times
- 3. If any edge still relaxes, there's arbitrage

Worked example

	USD	CAD	GBP
USD	1	0.8631	0.5903
CAD	1.1586	1	0.6849
GBP	1.6939	1.46	1

Exchange rates

Worked example

$$\text{product} > 1 \Leftrightarrow \Sigma\ -\log(\text{rate}) < 0$$

→ negative cycle detection

Take −log: product > 1 becomes a negative cycle

Worked example

```
USD→CAD→GBP→USD: 1.0013 > 1 ✓

Bellman–Ford still relaxing after n−1 passes → true
```

Bellman–Ford finds it

Java solution

```java
public class DetectArbitrage {
    public static boolean detectArbitrage(double[][] rates) {
        int n = rates.length;
        double[] dist = new double[n];
        java.util.Arrays.fill(dist, Double.MAX_VALUE);
        dist[0] = 0;
        for (int it = 0; it < n; it++) {
            boolean changed = false;
            for (int u = 0; u < n; u++) for (int v = 0; v < n; v++) {
                double w = -Math.log(rates[u][v]);
                if (dist[u] + w < dist[v] - 1e-12) { dist[v] = dist[u] + w; changed = true; }
            }
            if (!changed) return false;
        }
        return true;
    }
}
```

Complexity

- Time: O(n^3)
- Space: O(n^2)

Edge cases & pitfalls

- Use an epsilon
- Complete graph → n^3
- Start from any node

Interview tips & follow-ups

- Explain the log transform

Recap

- Multiplying rates becomes adding minus logs, so profit means a negative cycle, found by Bellman-Ford.
- Complexity: O(n^3)